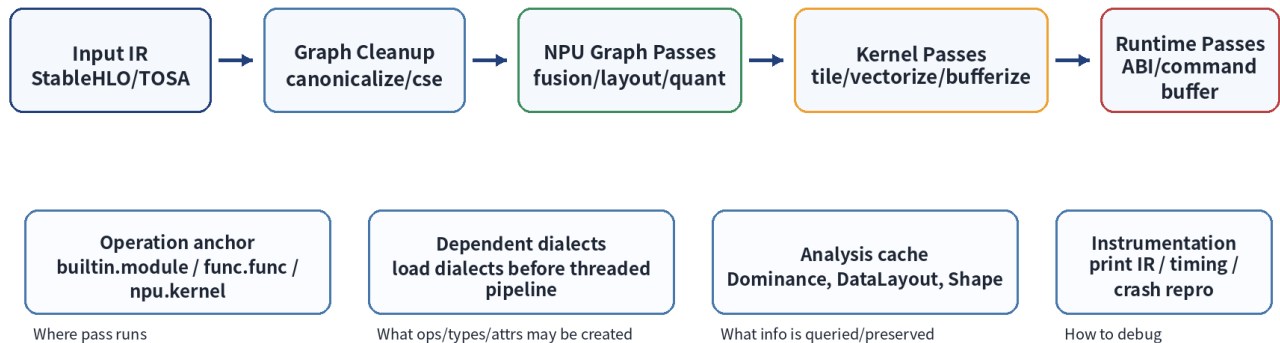


Lecture 09 - Pass Infrastructure

NPU Lowering Pass Pipeline Draft

Pass Infrastructure: NPU Lowering Pipeline



9강 산출물

MatMul+Bias+ReLU 예제를 대상으로 Pass ordering, legality boundary, analysis/preservation, debug/test strategy를 포함한 NPU lowering pass pipeline 초안을 작성한다.

1. 강의 목표

- MLIR pass 가 어떤 operation scope 에서 실행되고 어떤 IR mutation 을 허용하는지 설명한다.
- PassManager, OpPassManager, textual pass pipeline, nested pipeline 의 구조를 이해한다.
- NPU compiler 에서 fusion/layout/tile/bufferization/schedule/runtime emission 을 pass pipeline 으로 분리하는 기준을 세운다.
- MatMul+Bias+ReLU 예제에 대해 pass ordering, legality boundary, debug/test strategy 를 포함한 NPU lowering pipeline 초안을 만든다.

2. 왜 Pass Infrastructure 인가?

MLIR IR 을 읽는 능력만으로는 compiler 를 만들 수 없다. 실제 compiler 는 여러 개의 작은 pass 가 IR 을 점진적으로 정교화하고 target-specific IR 로 낮추는 구조로 동작한다.

NPU compiler 에서는 각 pass 가 성능과 correctness 에 직접 영향을 준다. Graph fusion 을 언제 할지, layout 을 언제 고정할지, bufferization 전후에 어떤 정보를 보존할지, DMA/barrier 를 어떤 시점에 넣을지가 모두 pass ordering 문제다.

Pass infrastructure 는 단순한 framework 가 아니라, 대형 compiler 를 디버그하고 테스트하고 재현 가능하게 만드는 product-quality interface 다.

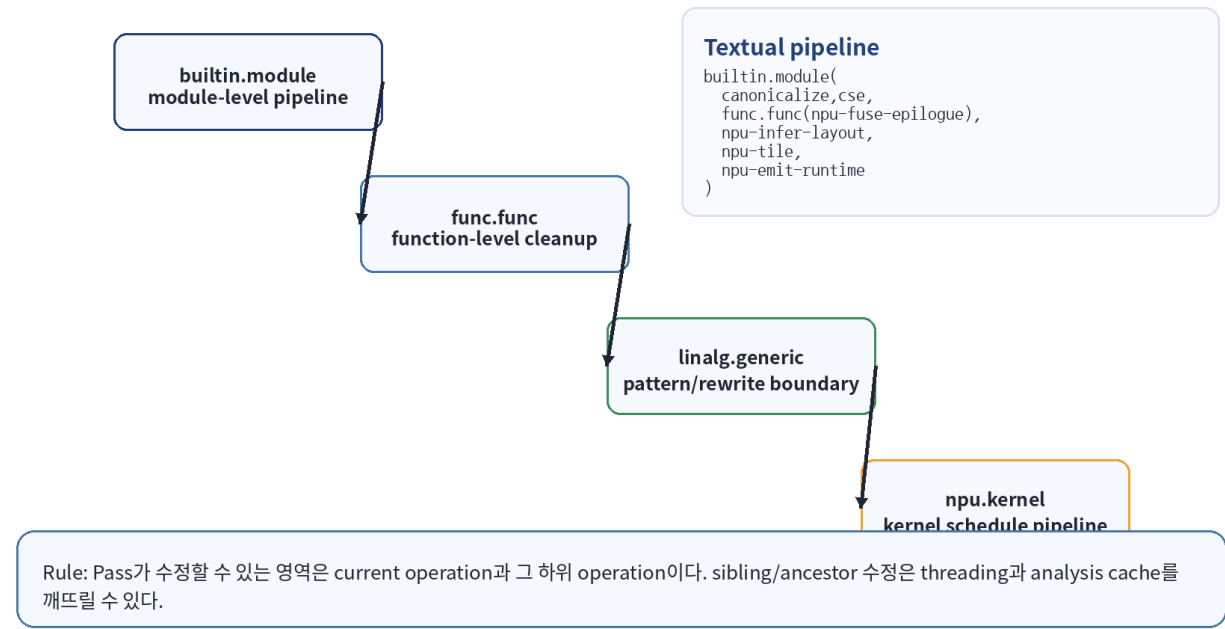
3. Pass 의 기본 Contract

MLIR 의 기본 transformation 단위는 operation 이다. Pass 는 current operation 을 기준으로 실행되며, 일반적으로 current operation 내부에 nested 된 operation 을 대상으로 분석/변환한다.

규칙	의미	NPU compiler 해석
Current operation 중심	pass 는 getOperation()으로 현재 anchor op 를 얻는다	module/function/kernel 중 정확한 anchor 선택
Sibling state 가정 금지	병렬 실행 중 sibling op 가 다른 pass 에서 처리 될 수 있다	다른 kernel command sequence 를 임의 수정 하지 않음
Global mutable state 금지	pass instance 가 복사/병렬 실행될 수 있다	target option 은 pass option 이나 analysis 로 전달
Copy-constructible	PassManager 가 병렬 실행을 위해 pass 를 복제 할 수 있다	cache 는 analysis manager 에 두고 pass 내부 mutable cache 최소화

4. Operation Anchor 와 Nested Pass Pipeline

Operation Anchoring and Nested Pass Managers



Textual pass pipeline 은 operation anchor 를 명시한다. 예를 들어 다음 pipeline 은 module 수준에서 canonicalize/cse 를 실행하고, func.func 내부에서 epilogue fusion 을 수행한 뒤 module 수준 layout inference 와 runtime emission 을 실행하는 구조다.

```
mlir-opt input.mlir \
  --pass-pipeline='builtin.module(
    canonicalize,cse,
    func.func(npu-fuse-matmul-epilogue),
    npu-infer-layout,
    npu-emit-runtime-abi
  )'
```

5. Pass 종류와 NPU 적용 위치

Pass type	Anchor 예	주요 목적	NPU 예
Module pass	builtin.module	symbol, ABI, whole-program legality	npu-emit-runtime-abi
Function pass	func.func	local graph rewrite/fusion	npu-fuse-matmul-epilogue
Kernel pass	npu.kernel	tile loop, DMA/barrier schedule	npu-insert-dma
Op-agnostic pass	any(...)	generic cleanup	canonicalize, cse
Interface pass	FunctionOpInterface	interface 기반 filtering	모든 function-like op 정리

6. C++ Pass Skeleton

아래 skeleton 은 function-level pass 로 MatMul epilogue fusion 을 수행하는 예시다. 실제 구현은 10 강 Rewrite Pattern 과 11 강 Dialect Conversion 에서 확장한다.

```
// Pseudo C++ skeleton for Lecture 09. Names are illustrative.
#include "mlir/Pass/Pass.h"
#include "mlir/IR/Diagnostics.h"
#include "mlir/IR/PatternMatch.h"
#include "mlir/IR/DialectRegistry.h"
#include "mlir/Dialect/Func/IR/FuncOps.h"
#include "mlir/Dialect/Linalg/IR/Linalg.h"

using namespace mlir;

namespace npu {
class NpuGraphDialect;
class NpuKernelDialect;
} // namespace npu

namespace {
struct FuseMatmulEpiloguePass
: public PassWrapper<FuseMatmulEpiloguePass,
  OperationPass<func::FuncOp>> {
  MLIR_DEFINE_EXPLICIT_INTERNAL_INLINE_TYPE_ID(FuseMatmulEpiloguePass)

  Option<bool> enableRelu{*this, "enable-relu",
    llvm::cl::desc("Fuse ReLU epilogue"),
    llvm::cl::init(true)};
  Statistic numFused{this, "num-fused", "Number of matmul epilogues fused"};

 StringRef getArgument() const final { return "npu-fuse-matmul-epilogue"; }
  StringRef getDescription() const final {
    return "Fuse MatMul + Bias + ReLU into an NPU graph candidate op";
  }

  void getDependentDialects(DialectRegistry &registry) const override {
    // TODO: registry.insert<npu::NpuGraphDialect>();
  }

  void runOnOperation() override {
    func::FuncOp func = getOperation();

    // TODO: query analyses if needed.
    // auto &dominance = getAnalysis<DominanceInfo>();

    bool sawIllegalPattern = false;
```

```

func.walk([&](linalg::MatmulOp matmul) {
    // TODO: match bias + activation users.
    // TODO: replace with npu_graph.matmul {epilogue = "relu"}.
    // ++numFused;
});

if (sawIllegalPattern) {
    func.emitError("illegal MatMul epilogue pattern for target NPU");
    return signalPassFailure();
}

// TODO: mark analyses preserved only if invariant is truly unchanged.
// markAnalysesPreserved<DominanceInfo>();
}
};
} // namespace

std::unique_ptr<Pass> createFuseMatmulEpiloguePass() {
    return std::make_unique<FuseMatmulEpiloguePass>();
}

// ... see example file for full skeleton ...

```

7. Dependent Dialects

Pass 가 어떤 dialect 의 operation/type/attribute 를 새로 생성할 수 있다면 그 dialect 를 dependent dialect 로 선언해야 한다. 특히 multi-threaded pass pipeline 에서는 pipeline 시작 전에 필요한 dialect 가 load 되어 있어야 하므로 이 선언이 중요하다.

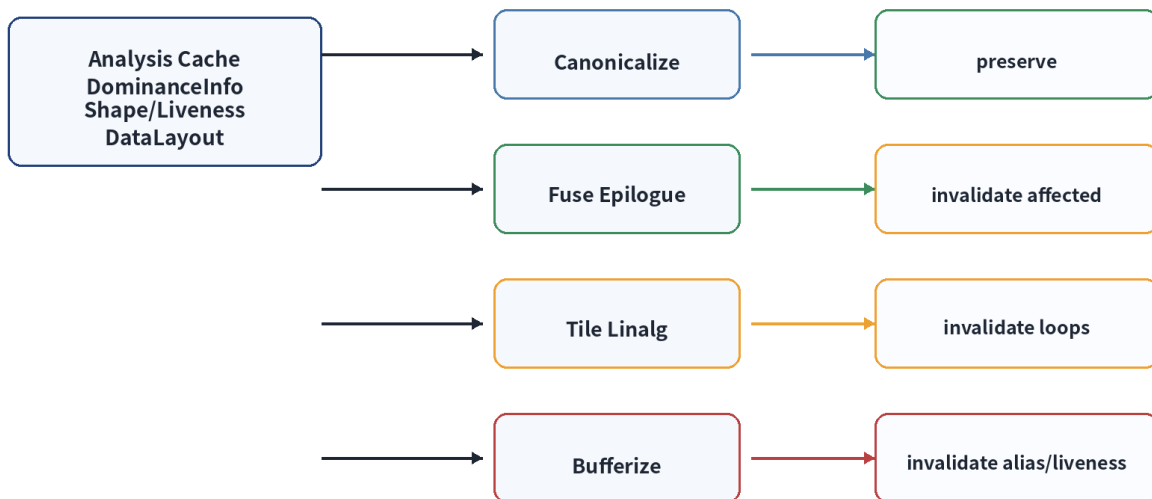
```

void getDependentDialects(DialectRegistry &registry) const override {
    registry.insert<npu::NpuGraphDialect,
                    npu::NpuKernelDialect,
                    memref::MemRefDialect>();
}

```

8. Analysis Management 와 Preservation

Analysis and Preservation: Compiler Performance Contract



NPU tip: expensive analyses such as layout feasibility, SRAM footprint, and bank-conflict checks should be cached, but only preserved when ti

Analysis 는 pass 가 필요로 하는 compiler fact 를 계산하고 cache 한다. 예를 들어 NPU compiler 에서는 shape info, layout feasibility, SRAM footprint, bank conflict, liveness, alias 정보를 analysis 로 만들 수 있다.

Analysis	사용 pass	Invalidation trigger	Preserve 가능 조건
DominanceInfo	fusion, scheduling	region/control-flow 변경	def-use/control-flow 보존
ShapeInfo	layout, tile	shape-changing rewrite	shape semantics 보존
LayoutFeasibility	layout inference	layout attr 변경	layout attr 불변
SramFootprint	tile, memory-space	tile size/buffer 변경	no-op pass
Liveness/Alias	bufferization, memory planning	value lifetime 변경	IR mutation 없음

9. Pass Failure 와 Diagnostic

Pass 가 target invariant 를 만족하지 못하면 조용히 잘못된 IR 을 만들지 말고 diagnostic 을 내고 signalPassFailure() 를 호출해야 한다. NPU compiler 에서는 unsupported dynamic shape, SRAM footprint overflow, illegal quantization scale, unresolved bank conflict 등이 failure path 다.

```
if (!target.supports(dtype)) {
  op->emitError("unsupported dtype for target NPU: ") << dtype;
  return signalPassFailure();
}

if (tileFootprintBytes > target.sramBytes) {
  op->emitError("tile footprint exceeds SRAM capacity");
  return signalPassFailure();
}
```

10. Options, Statistics, TableGen

Options 는 pass heuristic 을 command line 에서 재현 가능하게 만든다. Statistics 는 compiler transformation 의 결과를 수량화해서 regression 을 잡는다. TableGen Pass spec 는 summary, description, dependentDialects, options, statistics 를 선언해 boilerplate 와 문서 생성을 줄인다.

```
// Pseudo TableGen pass specification for Lecture 09.
include "mlir/Pass/PassBase.td"

def NpuFuseMatmulEpilogue : Pass<"npufusematmul-epilogue", "mlir::func::FuncOp"> {
  let summary = "Fuse MatMul + Bias + Activation into an NPU graph op";
  let description = [{
    Matches a linalg.matmul producer and epilogue consumers such as bias add and ReLU.
    Emits a target-aware NPU graph candidate op while preserving high-level tensor semantics.
  }];
  let dependentDialects = ["npufusematmul-epilogue"];
  let options = [
    Option<"enableRelu", "enable-relu", "bool", "true", "Fuse ReLU epilogue">,
    Option<"maxEpilogueOps", "max-epilogue-ops", "unsigned", "4", "Maximum epilogue ops to fuse">
  ];
  let statistics = [
    Statistic<"numFused", "num-fused", "Number of fused matmul epilogues">,
    Statistic<"numRejected", "num-rejected", "Number of rejected fusion candidates">
  ];
}
```

11. Pass Pipeline Registration

여러 pass 를 하나의 named pipeline 으로 등록하면 mlir-opt, unit test, product compiler 가 동일한 lowering recipe 를 공유할 수 있다.

```
// Pseudo pipeline registration.
#include "mlir/Pass/PassManager.h"
#include "mlir/Pass/PassRegistry.h"

using namespace mlir;

struct NpuLoweringPipelineOptions
: public PassPipelineOptions<NpuLoweringPipelineOptions> {
  Option<int> sramKb{*this, "sram-kb", llvm::cl::desc("SRAM size in KiB"), llvm::cl::init(512)};
  Option<int> tileM{*this, "tile-m", llvm::cl::desc("M tile"), llvm::cl::init(64)};
  Option<int> tileN{*this, "tile-n", llvm::cl::desc("N tile"), llvm::cl::init(128)};
  Option<int> tileK{*this, "tile-k", llvm::cl::desc("K tile"), llvm::cl::init(32)};
};

void buildNpuLoweringPipeline(OpPassManager &pm,
                             const NpuLoweringPipelineOptions &opts) {
  pm.addPass(createCanonicalizerPass());
  pm.addPass(createCSEPass());

  OpPassManager &funcPM = pm.nest<func::FuncOp>();
  funcPM.addPass(createNpuDecomposeCompositesPass());
  funcPM.addPass(createFuseMatmulEpiloguePass());

  pm.addPass(createNpuInferLayoutPass());
  pm.addPass(createNpuTileLinalgPass(opts.tileM, opts.tileN, opts.tileK));
  pm.addPass(createOneShotBufferizePass());
  pm.addPass(createNpuAssignMemorySpacePass(opts.sramKb));
  pm.addPass(createNpuInsertDmaPass(/*doubleBuffer=*/true));
  pm.addPass(createNpuPlaceBarriersPass());
  pm.addPass(createNpuEmitRuntimeAbiPass());
}

void registerNpuPipelines() {
  PassPipelineRegistration<NpuLoweringPipelineOptions>(
    "npu-lowering-pipeline",
    "Lower tensor/linalg IR to NPU runtime ABI",
    buildNpuLoweringPipeline);
}

// ... see example file for full pipeline registration ...
```

12. Instrumentation 과 Debugging

IR printing, timing, statistics, crash reproducer 는 NPU compiler bring-up 에서 반드시 켜야 하는 기본 debug interface 다. 특히 pass ordering 문제가 있을 때 --mlir-print-ir-after-change 와 --mlir-print-ir-after-failure 가 유용하다.

```
mlir-opt input.mlir \
  --pass-pipeline='builtin.module(func.func(cse,canonicalize),npu-lowering-pipeline)' \
  --mlir-print-ir-after-change \
  --mlir-print-ir-after-failure \
  --mlir-timing \
  --mlir-pass-statistics \
  --mlir-pass-pipeline-crash-reproducer=npu_fail.mlir
```

13. NPU Lowering Pipeline Draft

NPU Lowering Pass Pipeline Draft

Stage	Representative passes	IR boundary	Correctness check
Import	legalize-stablehlo/tosa, canonicalize	Tensor IR	shape/dtype preserved
Graph	fuse-matmul-epilogue, infer-layout	npu_graph	legal op set
Kernel	tile-linalg, vectorize, assign-memory-space	npu_kernel	SRAM fits
Memory	one-shot-bufferize, buffer-deallocation	memref	alias/lifetime valid
Schedule	insert-dma, place-barriers, bank-conflict-fix	SCF/Affine/NPU	dependency valid
Runtime	emit-descriptors, emit-command-buffer	npu_rt	ABI stable

Draft deliverable
각 pass마다 입력 dialect, 출력 dialect, 생성할 dependent dialect, 필요한 analysis, invalidation/preservation, FileCheck test를 명시한다.

9 강의 핵심 산출물은 pass 이름만 나열하는 것이 아니다. 각 pass 가 어떤 IR boundary 에서 동작하고, 어떤 legality 를 보장하며, 실패 시 어떤 diagnostic 을 내고, 어떤 FileCheck 테스트로 검증되는지까지 포함해야 한다.

Stage	Pass	Anchor	Input/Output	Required analysis	Correctness check
Import	npu-legalize-input	module	StableHLO/TOSA -> Tensor/Linalg	ShapelInfo	shape/dtype preserved
Graph	npu-fuse-matmul-epilogue	func.func	Linalg/Arith -> npu_graph	Dominance, Shape	legal fused op
Layout	npu-infer-layout	module	npu_graph attrs	LayoutFeasibility	no layout conflict
Kernel	npu-tile-linalg	func.func	Linalg -> SCF/Vector/NPU	Shape, TargetInfo	SRAM fits
Memory	npu-assign-memory-space	module	memref attrs	Liveness, Alias	no spill overflow
Schedule	npu-insert-dma	npu.kernel	NPU -> NPU scheduled	DependencyGraph	no hazard
Runtime	npu-emit-runtime-abi	module	npu_rt	SymbolTable	ABI fields valid

14. 실습 안내

- **Exercise A - Pipeline Annotation:** 01_pass_pipeline_input.mlir 과 pass pipeline string 을 읽고 anchor, pass order, input/output dialect 를 표시한다.
- **Exercise B - Pass Skeleton:** 03_npu_pass_skeleton.cpp 의 TODO 를 채우며 dependent dialect, option, statistic, signalPassFailure 위치를 표시한다.
- **Exercise C - NPU Pipeline Draft:** MatMul+Bias+ReLU 예제를 대상으로 8-12 개 pass 로 구성된 NPU lowering pipeline 을 작성한다.

15. Common Mistakes

- Pass 가 sibling 또는 parent operation 을 직접 수정한다.
- dependent dialect 를 누락해서 multi-threaded pipeline 에서 불안정하게 실패한다.
- IR 을 크게 바꿨는데 analysis preservation 을 과감하게 선언한다.
- Pass option 없이 hidden heuristic 을 사용해 성능/결과 재현이 어렵다.
- Bufferization 이후에 tensor-level graph fusion 을 시도한다.
- FileCheck 가 SSA 이름이나 printer detail 에 너무 의존한다.

16. References

- MLIR Pass Infrastructure: <https://mlir.llvm.org/docs/PassManagement/>
- MLIR Using mlir-opt: <https://mlir.llvm.org/docs/Tutorials/MlirOpt/>
- MLIR Passes: <https://mlir.llvm.org/docs/Passes/>
- MLIR Language Reference: <https://mlir.llvm.org/docs/LangRef/>
- MLIR Quickstart Rewrites: <https://mlir.llvm.org/docs/Tutorials/QuickstartRewrites/>